ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Laurea in Ingegneria Informatica

Exploring Phase-Change Memory as a Compute Accelerator Module:
A Virtual Prototype on the PULP Platform

Relatore:
Chiar.mo Prof.
Giuseppe Tagliavini

Presentata da:
Leonardo Domenicali

Invernale
2024/2025

Exploring Phase-Change Memory as a Compute Accelerator Module:

A Virtual Prototype on the PULP Platform

Leonardo Domenicali

Abstract

Traditional von Neumann architectures are hitting a wall. The physical separation between memory and processing creates a bottleneck that is becoming critical for data-intensive applications, particularly in AI and Machine Learning. Moving data back and forth between CPU and memory is becoming more time-consuming and energy-intensive than the actual computation. This issue is commonly referred to as the "memory wall" problem.

This dissertation explores Phase-Change Memory (PCM) as a potential solution through Analog In-Memory Computing (AIMC). The core idea is to perform the computation directly where the data is stored instead of relying on time-consuming data transfers. PCM is particularly suitable for this kind of application because its analog resistance states can naturally represent neural network weights, and its crossbar architecture allows for parallel computation of matrix-vector multiplications in a single analog operation.

The main contribution of this work is a virtual prototype of a PCM-based AIMC accelerator. The model is implemented in C++ and integrated into the GVSoC simulation platform within the PULP ecosystem. A significant part of the work focused on optimising the simulation itself, making it sufficiently fast to be useful for experimentation. This involved designing cache-friendly data structures and implementing multithreaded execution for the core Matrix-Vector Multiplication algorithm. Performance benchmarks demonstrate the effectiveness of these optimisations. The flat buffer data structure reduces data reads by up to 5 times compared to a standard multidimensional array approach, and multithreading provides significant speedup on multi-core systems, with optimal performance achieved at 8 threads for the tested configurations. The result is a configurable simulation framework that can be used to explore AIMC architectures and provides a foundation for co-designing hardware accelerators and the software needed to use them effectively.

Acknowledgements

TODO: Write acknowledgements.

Contents

List of Figures

List of Tables

List of Abbreviations

MVM	**M**atrix **V**ector **M**ultiplication
MM	**M**atrix **M**ultiplication
NVM	**N**on-**V**olatile **M**emory
PCM	**P**hase-**C**hange **M**emory
AIMC	**A**nalog-**I**n **M**emory **C**omputing
CIM	**C**omputing **I**n-**M**emory
CNM	**C**omputing **N**ear-**M**emory
AI	**A**rtificial **I**ntelligence
ML	**M**achine **L**earning
SoC	**S**ystem **o**n **C**hip
ADC	**A**nalog to **D**igital **C**onverter
DAC	**D**igital to **A**nalog **C**onverter

1

Introduction

1.1 Von Neumann Architecture and the Memory Wall

The field of computing is at a critical point. For decades, the industry has relied on the von Neumann architecture, which physically separates processing and memory units. While incredibly successful, this paradigm has created an inherent "memory wall" [Gho+24; WM95], where the time and energy spent moving data between the CPU and memory now dominate the computation cost. This bottleneck is especially evident in data-intensive fields like artificial intelligence and machine learning, which heavily rely on massive parallel operations, including Matrix-Vector Multiplication (MVM) and Matrix-Matrix Multiplication (MM) [Gho+24; Kha+24]. As Moore's Law—which predicts that the number of transistors on a chip doubles approximately every two years, leading to exponential growth in computing power—slows down, simply shrinking transistors is no longer a sustainable path to improving performance [TW17]. This paradigm shift underscores the urgent need for innovative computing architectures capable of sustaining progress in computational efficiency and scalability.

1.2 Dissertation Objective and Motivation

The "memory wall" has motivated a global research effort focused on new computing paradigms that move beyond the traditional von Neumann architecture. Among the most promising is Computing Near-Memory (CNM) and Computing In-Memory (CIM) [1.1], the idea of which is to perform computation directly where data is stored, eliminating the costs of data movement [Kha+24]. Among the CIM branches, Phase-Change Memory (PCM) has emerged as a possible candidate to enable Analog In-Memory Computing (AIMC) as its analog resistance states can naturally represent the weights of a neural network, and

the way the crossbar is configured allows for efficient analog and parallel computation.

However, designing and fabricating new hardware accelerators is a high-risk, high-capital process. Before committing to silicon, it is essential to have robust virtual platforms to explore architectural trade-offs, validate functionality, and co-design the necessary software. This project is motivated by the need for such a virtual prototype.

The primary objective of this dissertation is to implement and validate a virtual prototype of a PCM-based AIMC accelerator within the GVSoC [Bru+21] simulator for the Parallel Ultra-Low-Power (PULP) platform. PULP is an open-source hardware and software framework based on the RISC-V instruction set architecture (ISA), which aims to provide low-power computing systems.

To achieve this objective, two main goals were established:

- Develop a C++ model of a PCM accelerator capable of performing MVM operations, integrated as a memory-mapped peripheral in GVSoC.

- Investigate and implement performance optimisations for the simulation model to enable efficient and fast simulation even on limited hardware resources.

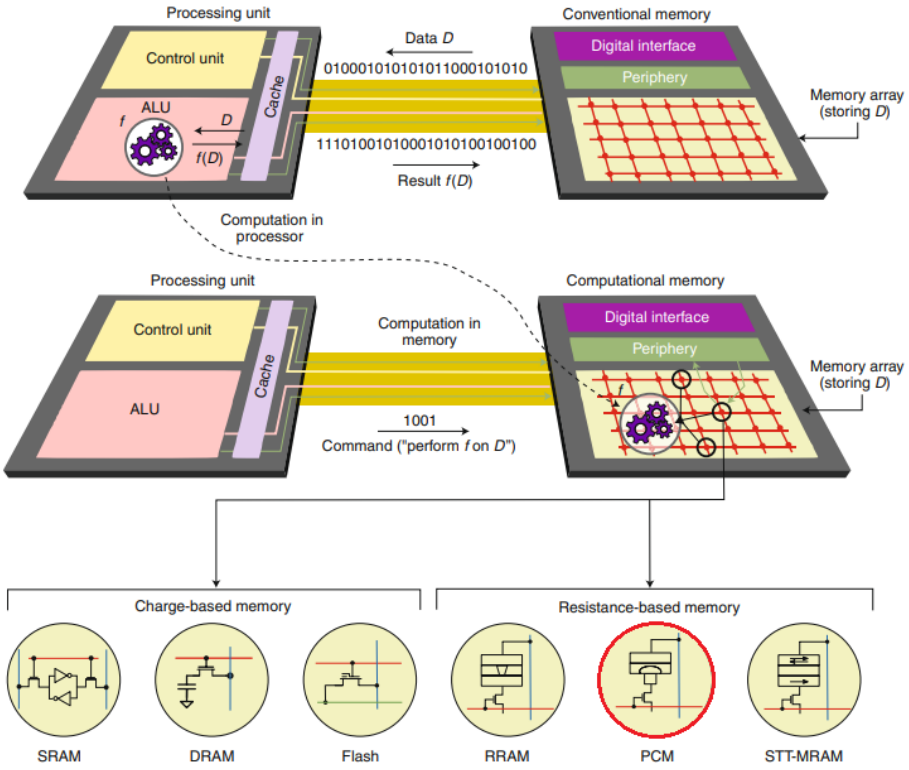


Figure 1.1: Computing In-Memory (CIM) architecture overview, moving computation closer to memory to alleviate the von Neumann bottleneck. Adapted from [Seb+20].

1.3 Summary of Contributions

This dissertation makes several contributions to the field of hardware acceleration and virtual prototyping for next-generation computing architectures. The primary outcomes of this research are the following:

- *A configurable virtual prototype of a PCM accelerator.* The core contribution is a fully functional C++ model of a PCM-based AIMC accelerator integrated into the GVSoC framework. This model is not just an abstract component but a working, configurable, peripheral that can be simulated within a complete System-on-Chip environment, serving as a valuable tool for architectural exploration.

- *Demonstration and analysis of simulation optimisation techniques.* This work demonstrates the effectiveness of applying advanced software optimisation principles to

hardware simulation. By implementing and benchmarking techniques such as flattened data buffers and multithreading, this project offers a strategy for significantly enhancing the performance and scalability of virtual prototyping, allowing for large-scale experiments even on platforms with limited computational resources.

2

Background

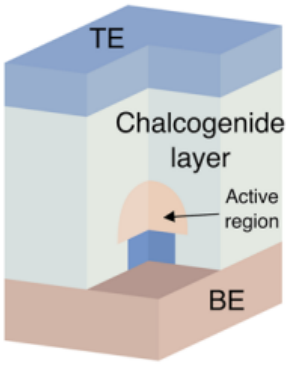
This chapter introduces the main concepts and technologies that are the basis of this dissertation.

We will start by introducing the Phase Change Memory (PCM), a non-volatile memory (NVM) capable of storing data by changing the state of the material, and why it is suitable for this kind of application. Then we will introduce the concept of Analog In-Memory Computing (AIMC) and how it can be used to speed up matrix-vector multiplication.
Finally, we will introduce the PULP platform and the GVSoC simulator as the tools and frameworks for the module implementation.

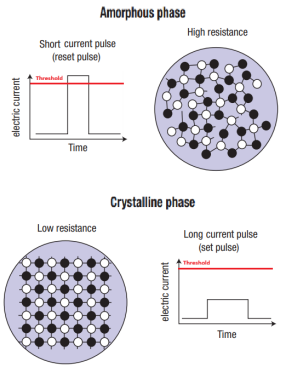
2.1 Phase Change Memory (PCM)

Phase Change Memory (PCM) [2.1a] is a type of NVM that uses the unique behaviour of some chalcogenide compound materials to switch between amorphous and crystalline states. These two phases offer differential electrical impedance that manifests as variable resistance [GS20; He+23]. By applying a current to the material, we can measure the output voltage, and by using Ohm's law, we can determine the resistance of the material and therefore the value of the cell. The state change is achieved by applying different heat levels to the material using electrical pulses [2.1b]: These pulses differ in both intensity and duration depending on the desired state. For this exact reason, we can notice a difference in write and read speed, with writes being slower and of variable durations but allowing reading speed comparable to DRAM read speed, or of around 50 ns [Sri+13].

From a literature review, we can find papers describing this innovation as early as the 1960s; however, in the following decade, the interest declined until the 2000s as drift-related issues emerged. In the early 2000s, the technology started to be developed for commercial use. One widely recognized implementation is the Intel Optane, a PCM-based technology called 3D XPoint [GS20; He+23]. Initially conceived as a binary memory technology, PCM has evolved to support multi-level cell architectures by using multiple levels of resistance to represent the stored data [Ant+24]. This property makes PCM particularly suitable for MVM and MM operations, as the weights of the matrices can be stored as resistance values directly within the memory cells.



(a) PCM Cell Schematics. Adapted from [Man+23]



(b) PCM Set and Reset pulses. Adapted from [WY07]

Figure 2.1: PCM Cell Schematic and Set/Reset Pulses.

2.2 Analog In-Memory Computing (AIMC)

Analog In-Memory Computing (AIMC) refers to a computing paradigm that aims to perform computations directly within the memory array, thus reducing the need for data movement between memory and processing units. This approach is particularly beneficial for operations that involve large amounts of data, such as matrix-vector multiplications (MVMs) and matrix-matrix multiplications (MMs), which are common in machine learning, AI, and scientific computing applications.
AIMC leverages the physical properties of memory devices, such as resistive switching in PCM, to perform computations in an analog manner. This characteristic allows for parallel processing of multiple operations by design, leading to significant speedups compared to traditional digital computing approaches. Moreover, analog capabilities scale better than digital operations, improving energy efficiency as the data grows.

2.3 Applying AIMC to PCM

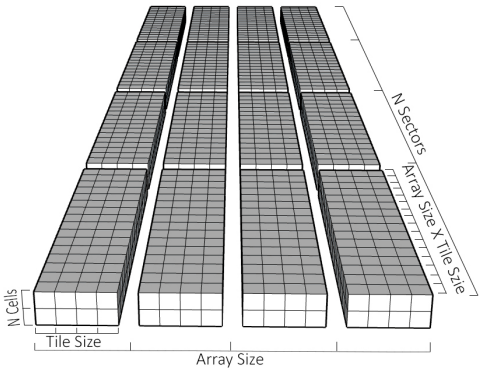
The PCM technology is particularly interesting for AIMC because it allows for the parallel execution of multiple operations. In fact, the internal structure of the PCM crossbar-array [2.2] allows for performing matrix-vector multiplications (MVMs) in a single step: by applying the input vector as current X_i to the rows of the PCM array and reading the output current from the columns, thanks to Kirchhoff's law, we retrieve the result $Y_i = \sum_{k=0}^{n} X_i \cdot R_{ik}$ in a single operation [He+23].

Moreover, the AIMC capability of a PCM crossbar makes good use of the reading operation, which is significantly faster than the writing operation optimising the overall performance for such computations. To apply this concept to a real PCM module, we need to consider some additional aspects, such as the presence of Digital-to-Analog Converters (DACs) to convert the digital input vector to analog signals for the rows and Analog-to-Digital Converters (ADCs) to convert the analog output currents from the columns back to digital values.

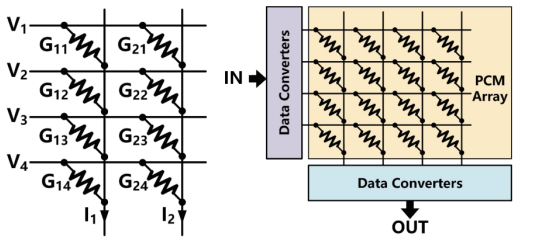
2.3.1 PCM Module Architecture

After considering the PCM cell architecture and its internal schematics, we can analyse the structure of a PCM module that can be used for AIMC. The PCM module architecture is shown in figure [2.2a].
The module is characterised by a matrix $R \times C$ of tiles. Each tile is a 3D array itself with surface $X \times Y$ and depth K, with K being the number of layers of PCM cells stacked

on top of each other. Each cell is able to hold n bits as variable resistance. The core concept of the entire module is to hold up to L layers, as $L = C \cdot K$, stored as $X \times J$ matrices, having $J = Y \cdot R$. During a computation, it is possible to select layers and matrix portions (called *sectors*) by providing the module with specific commands on the data bus. The tiles are interfaced with a set of DACs connected to the rows and ADCs connected to the columns, enabling bidirectional conversion of input vectors from digital to analog and output vectors from analog to digital, respectively. This is the basic architecture of the PCM module that we will use for our implementation.



(a) PCM Memory Architecture



(b) PCM Internal Schematics. Adapted from [He+23]

Figure 2.2:
PCM Module Architecture and Structure.

2.4 PULP Platform

Parallel Ultra Low Power (PULP) is an open-source computing platform developed by the PULP team at the University of Bologna and the ETH Zurich.

2.4.1 PULP Overview

The platform aims to deliver high-performance, energy-efficient computing solutions built on the RISC-V architecture. It features a variety of processing units, including RISC-V cores and specialized accelerators. The design of a computing platform based on these components is highly configurable, enabling designers to customize the architecture to meet specific application needs. PULP is especially suitable for edge computing applications,

where power consumption and performance are crucial.

2.4.2 GVSoC Simulator

The PULP ecosystem is supported by a complete software stack, including the SoC simulator (GVSoC) [Bru+21], which provides an event-driven simulation environment for PULP-based systems. The simulator allows developers to model and simulate the behavior of systems-on-chip, enabling evaluation of performance, power consumption, and other metrics before hardware deployment.

GVSoC supports multi-core processing, memory hierarchies, and peripheral devices. Custom modules are implemented as C++ classes that communicate via a bus-based wire system and can be memory-mapped to specific address ranges. Each module is configured through JSON files that define its parameters and connections to other components. The following chapter describes how these mechanisms are used to implement the PCM accelerator model.

3

PCM Module Implementation

This chapter will focus on the implementation of the PCM module within the GVSoC framework. We will start by introducing the architecture of the module and the data structures used to store the crossbar values. Then we will analyse the MVM algorithm and how we can optimise it to achieve better performance. Before starting the implementation is key to define how the simulator works and the constraints of the architecture and the software running the simulator.

3.1 GVSoC Integration

To integrate the PCM module into GVSoC, we implemented it as a memory-mapped peripheral following the standard GVSoC module architecture.

3.1.1 Module Structure

The implementation consists of three components:

C++ Model Class: The core logic is implemented in a class inheriting from *vp::component*. This class handles incoming memory requests, maintains the PCM crossbar state, and executes MVM operations.

Python Generator: A configuration script generates the module's JSON configuration file and defines its connections to other system components. The generator specifies the module's address range, memory size, and available operational modes.

JSON Configuration: The generated JSON file contains all module parameters including matrix dimensions, cell precision, and timing characteristics.

3.1.2 Bus Interface

The module connects to the system through GVSoC's wire mechanism. A request handler with signature *vp::IoReqStatus handler(vp::Block* block, vp::IoReq* req)* processes incoming read and write operations from the simulated RISC-V cores.

The memory map exposes three regions:

- **Weight Memory:** Direct access to PCM cell values for reading and writing weights values.
- **Input/Output Vectors:** Buffers for MVM operands and results.
- **Control Registers:** Configuration of operation mode, matrix dimensions, and computation triggers

3.1.3 Timing Model

GVSoC is a event driven simulator, the timing is represented in two ways:

- **Functional Timing:** Each operations (e.g., read, write) can be performed in zero time, meaning that the operation is completed as soon as it is called but a latency is added to the envent to guarantee correct timing simulation.
- **Event Scheduling:** Operations can be scheduled to occur after a specific delay by enqueing the event in the simulator's event queue. Once the delay elapses, the event triggers the module's *ClockEvent* handler to process the operation.

The module will make good use of these mechanism, simulating the timing characteristics of PCM operations.

3.1.4 Power Management

The module connects to GVSoC's power wire system, allowing it to be powered down when not in use. While no specific power management features are implemented, the module can be easily extended to include them in the future.

3.2 Architecture Constraints

Before analysing the module itself, it is important to define some hardware constraints of the hardware running the simulator and software constraints of the simulator itself.

3.2.1 Hardware Constraints

The hardware running the simulator spans various architectures and configurations, from low-power laptops to HPC nodes and high-power workstations. This is a major concern when writing optimised code, especially when micro-optimisations are considered. For this reason, the optimisation will be maintained at a high abstraction level, avoiding architecture-specific features and the adoption of hardware acceleration.

3.2.2 Software Constraints

The GVSoC framework is a C++ based SoC event simulator. While it supports many architectures and configurations, the software is intentionally single-threaded to preserve event-accurate simulations. Simulations involving multiple clusters on HPC nodes and high-performance workstations reveal that the main bottlenecks are single-thread performance and memory consumption. Therefore, the proposed optimisations will target both CPU efficiency and RAM usage.

3.3 PCM Module Architecture

The PCM module architecture is shown in Figure [2.2a] in the previous chapter.

To summarise, the module is characterised by a matrix of tiles. Each tile is a 3D array itself with surface $X \times Y$ and depth K, with K being the number of layers of PCM cells stacked on top of each other. Each cell is able to hold n bits as variable resistance. The tiles are interfaced with a set of DACs connected to the rows and ADCs connected to the columns, enabling bidirectional conversion of input vectors from digital to analog and output vectors from analog to digital, respectively. This is the basic architecture of the PCM module used in this implementation.

3.4 Module Data-Structures

Based on the architecture of our module, we notice the need for a large memory bank capable of accommodating all the values. To add an abstraction layer, we can model only the layers and matrix sectors without major concern for the physical layer. The key challenge lies in defining a data structure to store all the weights within the matrix. Dynamically allocated arrays appear to be the best option, as they allow for a fully configurable design.

Then, it is crucial to define a matrix-vector multiplication algorithm to test the effec-

tiveness of the data structure.

$$Y_{s \cdot J+j} = \sum_{i=0}^{I-1} (\sum_{l \in L_s} M_{s,l,j,i} \cdot X_i) \tag{3.1}$$

Where:

s = sector index
j = tile row index
J = number of rows in the tile
l = layer index
i = column index
L_s = layers used in the sector
I = number of columns in the matrix
M = matrix
X = input vector
Y = output vector

The above equation describes the MVM operation performed by the PCM module. It iterates over each enabled sector s and for each row j of the tile, computes the dot product between the input vector X and the corresponding row of the matrix M across all enabled layers L_s. We use this naive implementation to perform initial benchmarks and determine the best data structure.

3.4.1 Data-structure Optimisation

The first step is to define a general data structure that can be used to store the values of the PCM module.

4D Array

An immediate solution is to use a 4-dimensional dynamically allocated array. This architecture allows for easy access to each cell using a standard n-dimensional matrix approach. However, it is not optimal due to significant performance issues. Dynamically allocated multidimensional arrays can create cache locality problems, which may lead to more cache misses and slow down the algorithm. Additionally, there is considerable overhead associated with pointer allocations and deallocations during the lifespan of the model object. Another critical concern for this use case is suboptimal memory usage. Each allocation may not be contiguous in memory, resulting in fragmentation and inefficient use of available memory. Furthermore, pointer dereferencing and pointer arithmetic can lead to inefficient memory access patterns, especially with large matrix dimensions.

Flat Buffer

A second, more optimized approach would be to use a "flat" n-dimensional array. This solution requires more advanced indexing calculations, leading to significantly fewer instructions for memory allocation and deallocation. Implementing this approach results in noticeable performance improvements. Analysis of a test program using Cachegrind (a Valgrind tool that simulates cache access) shows a marked decrease in cache misses and a reduction in memory reads. The decrease in cache misses is due to the sequential storage of data, which is read from memory in a similar sequential manner. The reduction in memory reads occurs because contiguous memory allocation eliminates the overhead of pointer dereferencing. One of the main drawbacks of this method is the need to manually compute the index of each cell. However, using inline functions, macros, and strategies for register reuse can help reduce complexity and the number of instructions required.

3.4.2 Differential Algorithm

Before focusing on specific optimisations for the MVM algorithm, it is important to note that the PCM module is capable of using differential weights. This means that each weight is represented by two PCM cells, one for the positive part and one for the negative part: the final weight is computed as the difference between the two cells. This approach allows for maintaining high precision despite the inevitable drift caused by the module's physical characteristics. To accommodate this feature, without changing the data structure or the MVM algorithm, it's possible to use two computation passes.

$$Y_{s\cdot J+j} = \sum_{i=0}^{I-1}(M_{s,h_l,j,i} - M_{s,l_l,j,i}) \cdot X_i = \sum_{i=0}^{I-1} M_{s,h_l,j,i} \cdot X_i - \sum_{i=0}^{I-1} M_{s,l_l,j,i} \cdot X_i \tag{3.2}$$

Where:

h_l = High Layer
l_l = Low Layer

This approach, while doubling the number of operations required, allows us to maintain the same data structure and MVM algorithm. This trade-off is acceptable because the differential approach is not commonly used and the optimisations for one of the two cases are mutually beneficial.

3.5 MVM Algorithm

In this section, we analyse and improve the MVM algorithm performance.

3.5.1 Standard Implementation

The standard algorithm to compute an MVM using this PCM module is already described in the previous section [3.1]. While some faster algorithms for sparse [Abb+23] and band matrix are described in the scientific literature, it is notable that, in this generalised case, the matrix does not adhere to any of those rules, and keeping track of the non-zero elements would require additional memory, which is not optimal for our use case. For this reason the optimisations will target the standard algorithm by working on the code and structures.

Looking at the data structure, we notice two main points. First of all, the matrix is already divided into blocks (tiles) and sectors. As suggested by some studies [YSV15], granularity is one of the keys for fast MVMs and MMs algorithms. Secondly, our MVMs compute row by vector, rows that are stored sequentially in our flat buffer, meaning our data already follows cache-friendly and granularity rules. This implementation already offers performance speed-ups while closely simulating the PCM module.

3.6 Optimisations

While this step is not necessary in a simulation environment, it is crucial to run big simulations in a reasonable time, even on low-power devices, enabling wider accessibility and usability.

3.6.1 Improved Cache-friendliness

From the analysis of the standard implementation [3.1], there is still some potential for improvement in the cache friendliness. In particular, the way layers and sectors are handled can be aligned to improve cache-friendliness.

$$Y_{s \cdot J+j} = \sum_{l \in L_s} (\sum_{i=0}^{I-1} M_{s,l,j,i} \cdot X_i) \tag{3.3}$$

Improving cache locality can be achieved by rearranging the layers to be processed in the outer loop. By fully computing one layer before moving on to the next, we can access data in a more sequential manner. This approach results in fewer cache misses and reduces the number of data reads. It is important to consider this design change when designing the data structure, particularly regarding how layers and sectors are loaded into memory.

3.6.2 Register Reuse

Register reuse strategies are a great way to improve performance by reducing the number of memory accesses. Specifically to this case, computing the sum of each row into a register

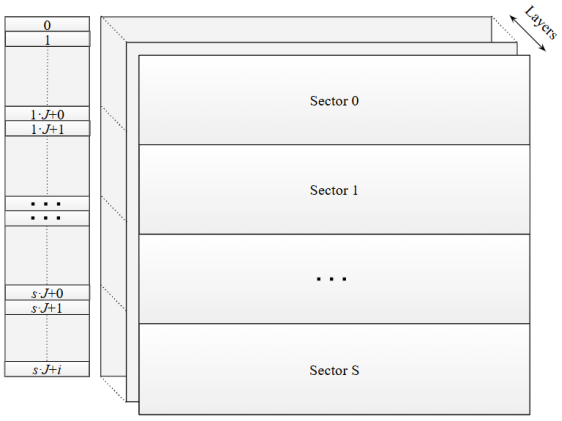


Figure 3.1: PCM matrix data-structure after cache-friendliness optimisation.

without the need to store intermediate results in the output vector eliminates unnecessary memory writes and reads.

To implement this optimisation, a temporary variable is used to store the result of each row computation before writing it back to the output vector. This approach leads to a significant reduction in memory accesses, with the only drawback that it is applicable only when the compiler is set to optimise the code. Analysing the assembly code generated for each of the cases, it is immediately clear why. In the non-optimised case the compiler store that value on the stack and retrieves it at each iteration degrading the overall performances.

3.6.3 Multithreading

While cache-friendliness and register reuse are great ways to improve performance, multithreading is one of the best performance boosters in both MVMs and MMs. [YSV15; DAm+16; Tan+24; SG98]. To both take advantage of multithreading, while minimising thread synchronisation and race conditions, which would slow down the algorithm, a modified implementation is proposed by moving some of the inner loops to different threads. The loops in question are the sector loops. This approach guarantees granularity and, because each result value is stored at a different vector index, there is no need to synchronise processes. It is possible to increase the thread count beyond the number of sectors by splitting the row loop, but this requires proper synchronisation. In this case, atomic operations are used; the overhead is minimal because the atomic operations are limited to the write operation of the output vector; however, the performance impact is still present.

While implementing multithreading is fairly straightforward, the problem lies in using

Optimised Function Entry

```
mov    r10, rdi
push   r12
lea    r9, [rcx+4]
push   rbp
mov    rbp, r8
push   rbx
mov    rbx, rdx
```

Un-Optimised Function Entry

```
push   rbp
mov    rbp, rsp
push   rbx
sub    rsp, 120
mov    QWORD PTR [rbp-88], rdi
mov    QWORD PTR [rbp-96], rsi
mov    QWORD PTR [rbp-104], rdx
mov    QWORD PTR [rbp-112], rcx
mov    QWORD PTR [rbp-120], r8
```

Optimised Variable Access

```
movsx  rdx, edi
mov    r8, QWORD PTR [rbx+rdx*8]
```

Un-Optimised Variable Access

```
mov    eax, DWORD PTR [rbp-24]
cdqe
lea    rdx, [0+rax*4]
mov    rax, QWORD PTR [rbp-64]
add    rax, rdx
mov    eax, DWORD PTR [rax]
mov    DWORD PTR [rbp-68], eax
```

Table 3.1: Comparison between optimised and un-optimised function setup and variable access. Generated using Compiler Explorer [God].

the right number of threads. Some studies suggest that the number of threads should be equal to the number of physical cores [BS24]: Using more threads will lead to "over-threading" and, therefore, worsen the performance. However, other studies suggest that the number of threads is, in small variation, independent of the number of physical cores and, especially in cases where synchronisation is not required, therefore suggesting that a different number of threads could be beneficial [NK11]. To explore this aspect, algorithm tests are proposed to determine the optimal number of threads for specific cases. The goal is to achieve results that are as general as possible for consumer hardware.

3.7 Digital to Analog & Analog to Digital Converters

3.7.1 DAC

The Digital to Analog Converter (DAC) is a key component of the PCM module, responsible for converting digital input vectors into analog signals that can be applied to the rows of the PCM array. However, there is no need to model the actual analog signal because the PCM model, being simulated on a digital platform, can directly work with digital values. It is important to note that the DAC introduces latency that is annotated as part of the simulation.

3.7.2 ADC

The Analog to Digital Converter (ADC) is another key component of the PCM module, responsible for converting the analog output signals from the columns of the PCM array back into digital values. Similar to the DAC, there is no need to simulate the actual analog signal because the PCM model can directly work with digital values; however, the ADC will clip the output values to the maximum and minimum values representable by the number of bits used in the conversion. To model this behaviour, the ADC model applies a clipping function to the output values.

$$n = \textit{output bit size} \tag{3.4}$$

$$ADC(y) = \begin{cases} 2^{n-1} - 1 & \text{if } y > 2^{n-1} - 1 \\ -2^{n-1} & \text{if } y < -2^{n-1} \\ y & \text{otherwise} \end{cases} \tag{3.5}$$

This function ensures that the output values are always within the valid range for the given output bit size. Similar to the DAC, the ADC introduces latency that is annotated as part of the simulation.

3.8 GVSoC Integration

To integrate the PCM module into the GVSoC framework, we need to create a new peripheral class that represents the PCM. This class will be responsible for handling the communication between the model and the rest of the system, as well as managing the internal state of the PCM. The PCM is implemented as a memory-mapped peripheral, allowing the processor to read and write data to the PCM using standard memory access instructions. The module is connected to the system bus, allowing it to communicate with other peripherals and memory components in the system. The bus interface has been implemented using the standard GVSoC wire.

The module has a defined mapping capable of exposing the memory space, read and write of the PCM cells, and the main AIMC unit, allowing the processor to store the input vector, trigger the MVM operation, write the input vector, and read the output vector. All the operations expose the result immediately by queuing the result at the correct time; the only exception is the MVM operation, which, while taking place immediately, exposes the result after a defined latency once the event timer signals the module. The PCM module also includes a set of configuration registers that allow the processor to configure the operation of the PCM module, such as the MVM dimensions, weights precision, and

different MVM modes like signed and unsigned, or the previously described differential mode.

4

Tests and Results

This chapter presents the results of the tests conducted and the module validation.

4.1 Algorithm Analysis and Tests

4.1.1 Data Metrics

The project aims to provide fast and efficient MVMs for matrices stored in PCM memory up to 2 layers of 512 $\times$ 512 with 8 bit weights and 8 bit integer inputs. For this reason, experiments consider matrix dimensions up to 512 $\times$ 512. All tests were executed after a clean start-up to minimise noise given by other applications for context switching and synchronization inside the CPU scheduler. The results come from batch runs of 100 pseudo-random matrices sequentially computed with different algorithms and different setups. This approach reduces the noise given by comparing different matrix configurations and focuses on the performance of the algorithm itself. We conducted tests using two profiles, one with optimisation switched on and the other off, to see how compiler optimisations affect the results.

Performances Data

The main metric used to evaluate the performance of the algorithms is the time to complete a single MVM computed with the aid of the $std :: high\_resolution\_clock$.

Table 4.1: Cachegrind Performance Counters for Optimised MVM Algorithms (-O3)

Algorithm	**Instructions (Ir)**	**Data Reads (Dr)**	**L1 Read Misses**	**Data Writes (Dw)**	**Branches (Bc)**	**Branch Misses**
Naive	8,132,735	2,623,013	8660	263,186	1,049,093	534
Naive Flat	7,343,702	1,311,265	8270	262,154	1,049,093	534
Optimise Flat	1,814,663	66,586	8340	1027	33,809	1077
4-Thread	1,814,672	66,580	8346	1024	33,804	1071
8-Thread	1,808,472	65,544	8211	0	33,792	1050
16-Thread	1,807,632	65,552	8232	0	33,808	1066

Table 4.2: Cachegrind Performance Counters for Unoptimised MVM Algorithms (-O0)

Algorithm	**Instructions (Ir)**	**Data Reads (Dr)**	**L1 Read Misses**	**Data Writes (Dw)**	**Branches (Bc)**	**Branch Misses**
Naive	51,383,891	20,973,083	8653	786,957	1,049,609	554
Naive Flat	33,820,273	14,681,640	8260	1,311,250	1,049,609	555
Optimised Flat	8,419,725	4,730,004	8335	3126	526,361	1070
4-Thread	8,419,688	4,729,980	8338	3140	526,356	1064
8-Thread	8,410,232	4,725,784	8200	5168	526,344	1055
16-Thread	8,411,440	4,726,864	8214	5248	526,352	1061

Figure 4.1:
Violin plots of the results with and without optimisation.

4.1.2 Performance Analysis

The benchmark results demonstrate that the optimised implementation achieves approximately 4× speedup over the naive approach when compiled with the O3 flag. This performance gain derives from the combined effect of algorithmic improvements and compiler optimisations.
The following sections analyze each optimisation technique individually to understand each contribution.

Impact of Compiler optimisations

Figure 4.1 shows the execution time distribution for different algorithm variants compiled with both O0 and O3 flags. The compiler optimisation level has a great impact on all implementations, with O3 producing on average 2× faster code across all tested algorithms. Using O3, even the naive implementation benefits from compiler optimisations, though it remains the slowest one. The only exception takes place with very small matrices, where the multithreaded overhead exceeds the benefit of parallelization. Notably, compiler optimisation interacts differently with our algorithmic improvements depending on the optimisation level. Under O0, the 8-thread implementation outperforms the optimised single-threaded version, as our hand-coded optimisations cannot compensate for the lack of compiler support. However, under O3, the single-threaded optimised version becomes the fastest option, as the compiler can make good use of the optimisations we implemented through improved data structures and memory access patterns.

Cache Behavior

Tables 4.1 and 4.2 present detailed performance counters collected using Valgrind's Cachegrind tool. These metrics provide more data on how each algorithm interacts with the CPU cache hierarchy. Starting with the instruction counts (Ir), we observe a 4× reduction in instructions executed when compiled with O3 and up to 6× without compiler optimisations. This outcome aligns with the overall memory access reductions seen in both data reads (Dr) and writes (Dw). The cache miss rate (L1 Read Misses) remains constant across all algorithms and optimisation levels, indicating that our optimisations primarily reduce the volume of memory traffic rather than improving cache hit rates. However, a closer look at the miss count reveals that the remaining misses are the result of compulsory misses that Cachegrind can not simulate away by including hardware prefetching simulation. Using the Linux perf tool, further analysis shows that the algorithms achieve an almost-zero L1-cache miss rate on typical consumer hardware. While this tool might not be as accurate as Cachegrind, it gives a better idea of the real cache miss rate, counting the hardware prefetcher.

Memory Access Patterns

The register reuse optimisation shows great benefits in memory usage. Under O0 compilation, data reads decrease by a factor of 4× compared to the naive implementation, while under O3 this improvement reaches 40×. More significantly, write operations are reduced by 250× (O0) and 260× (O3). This reduction in memory traffic directly translates to performance gains, as memory accesses are the bottleneck in MVM operations. The difference between reads and writes between O0 and O3 highlights how compiler optimisations can further enhance the benefits of algorithmic improvements.

Multithreading Performance

The effectiveness of multithreading can vary greatly depending on the optimisation level. Under O0, the 8-thread implementation outperforms the single-threaded version by approximately 2× on heavier workloads. This is because, without compiler optimisations, the benefits of parallel execution outweigh the overhead of thread management. However, under O3, the single-threaded optimised version becomes the fastest solution, outperforming any multithreaded implementation. This shift occurs because the compiler greatly enhances the effectiveness of the single-threaded optimisations. Cachegrind results indicate that the multithreaded versions consume about the same amount of memory accesses as the single-threaded optimised version.

However, studies show that multithreaded implementations can incur additional memory overhead [Tan+24]. Given the memory-constrained nature of large-scale SoC simulations discussed in the previous chapter (Section 3.2.2), this overhead cannot be ignored.

Based on these findings, the final implementation uses a dynamic dispatch approach:

- When compiled with O3 optimisations: always use the single-threaded optimised version
- When compiled with O0: select algorithm based on matrix setup and size
- Optional compile-time flag to force multithreading for specific use cases

To further enhance configurability, the Python generator exposes the thread count as a tunable parameter, allowing users to select the number of threads based on specific workload characteristics and system capabilities.

Compiler Code Generation Analysis

Further analysis of the generated assembly code under the O3 flag provides more insight into how the compiler optimisations contribute to performance improvements.

Vectorization The compiler automatically recognizes vectorizable loops and generates SIMD instructions to process multiple data elements in parallel based on the data layout and hardware resources. The specific instructions used depend on the target architecture; for the tests conducted, the X86 SSE2 flags were correctly applied.

Loop Unrolling One of the most significant optimisations observed is loop unrolling. By unrolling loops, the compiler reduces the overhead of loop control instructions and increases instruction-level parallelism. The specific unrolling factor, in the tested cases, is 16 with registers of 128-bit size used to process 16 8-bit integers in parallel.

Register Allocation The compiler maintains the majority of the variables in registers during the computation, minimizing stack accesses, thus reducing memory traffic.

4.2 Module Validation

To validate the module, we conducted tests on the GVSoC platform, using dedicated scripts to check the correctness of the results.

4.2.1 Algorithm Validation

To validate the algorithm, we performed two tests. The first one is a simple test with a known matrix and vector; the result of the MVM is known and can be compared with the result of the algorithm. The second required randomly generated matrices and vectors; the result of the MVM is computed by the algorithm, and configuration files are generated to be used in a Python script capable of computing the same MVM using NumPy as a reference. The results of the two tests were compared to produce a pass or fail result. As the algorithm is both deterministic and only uses integer arithmetic, the results are expected to be exactly the same without any variance. After thorough testing, the algorithm was validated, passing all tests.

4.2.2 GVSoC Integration Validation

To validate the integrations with GVSoC, different tests were performed. To maintain the scope of the project, the value of the weights was limited to 4 bit integers; however, the module is fully parametric and can be used with any number of bits. The clipping of the weights was the first aspect to be validated; tests with weights out of range were run, and the result of the MVM was checked to be correct. Then tests with randomised vector and matrix values with all ones weights were executed, the result of the MVM is known and can

be easily checked as $Y_i = \sum_i^N 1 \cdot X_i$, making the expected result appear on every output value.

Finally, a test with random weights and vectors containing the sign of the weights was conducted, as before, the result of the MVM was easily checked as the sum of the values of the rows with the sign of the weights. To further validate the module, using the same approach as before, a value within each row of the matrix was set to zero, and the result was checked by comparing the difference, given by excluding the zeroed value, with the expected value. All tests passed. The module correctly handles edge cases like weight clipping and produces results identical to those of NumPy reference implementations.

5

Conclusion & Future Work

5.1 Conclusion

In this dissertation, we designed, implemented, and validated a virtual prototype of a Phase-Change-Memory-based analog in-memory computing accelerator within the GVSoC simulation environment for the PULP platform. The core of the work is a C++ virtual prototype that performs efficient matrix-vector multiplication by modeling a PCM crossbar array, abstracting its core operational principles rather than its deep device physics, and incorporating some key non-ideal effects like ADC clipping and wheights size. This model was seamlessly integrated into GVSoC as a memory-mapped peripheral, which allows full-system simulation and enables the accelerator's performance to be evaluated within a complete system-on-chip environment. To improve simulation speed and scalability, we flattened data structures to optimise memory-access patterns, reducing cache misses and increasing overall throughput, and we introduced multithreaded kernels that parallelise the MVM computations across cores. Extensive benchmarking identified the optimal thread counts for a variety of hardware configurations. The final accelerator model is highly configurable: users can adjust matrix sizes, cell sizes, and threading options, making the prototype a valuable tool for exploring architectural trade-offs in analog in-memory computing systems.

5.2 Future Work

While this dissertation has laid a solid foundation, there are several ways for future work that can further enhance the PCM-based AIMC accelerator model and its applications:

Extending the PCM Model: Future work could focus on incorporating more detailed physical models of PCM devices, including variability, endurance, and retention characteristics. While the current model it's already capable of simulating drift resistance MVM by using the differential algorithm, a well-known effect of PCM devices is the variability of read operation while under thermal stress. The current model exposes read, write, and computation counters that could be used for further analysis of these effects.

Exploring Additional Optimisation Techniques to Improve Simulation Performance: Further research could investigate additional optimisation strategies. The current data structure and the module itself could be further optimised by exploring both SIMD instructions and GPU acceleration. Without the need to move the PCM weights frequently, this module is a perfect candidate for GPU acceleration.

Bibliography

[WM95] Wm. A. Wulf and Sally A. McKee. "Hitting the memory wall: implications of the obvious". en. In: *SIGARCH Comput. Archit. News* 23.1 (Mar. 1995), pp. 20–24. ISSN: 0163-5964. DOI: 10.1145/216585.216588. URL: https://dl.acm.org/doi/10.1145/216585.216588 (visited on 11/02/2025).

[SG98] P.D. Sulatycke and K. Ghose. "Caching-efficient multithreaded fast multiplication of sparse matrices". In: *Proceedings of the First Merged International Parallel Processing Symposium and Symposium on Parallel and Distributed Processing*. ISSN: 1063-7133. Mar. 1998, pp. 117–123. DOI: 10.1109/IPPS.1998.669899. URL: https://ieeexplore.ieee.org/document/669899 (visited on 04/18/2025).

[WY07] Matthias Wuttig and Noboru Yamada. "Phase-change materials for rewriteable data storage". en. In: *Nature Mater* 6.11 (Nov. 2007). Publisher: Nature Publishing Group, pp. 824–832. ISSN: 1476-4660. DOI: 10.1038/nmat2009. URL: https://www.nature.com/articles/nmat2009 (visited on 09/12/2025).

[NK11] Alexandru Nicolau and Arun Kejariwal. "How Many Threads to Spawn during Program Multithreading?" In: *Languages and Compilers for Parallel Computing*. Ed. by Keith Cooper, John Mellor-Crummey, and Vivek Sarkar. Vol. 6548. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 166–183. ISBN: 978-3-642-19594-5 978-3-642-19595-2. DOI: 10.1007/978-3-642-19595-2_12. URL: http://link.springer.com/10.1007/978-3-642-19595-2_12 (visited on 04/19/2025).

[Sri+13] Rajarajan Srinivasan et al. "A Study on the Challenges and Prospects of PCM Based Main Memory Architectures". In: *Middle East Journal of Scientific Research* 18 (Dec. 2013), pp. 788–795. DOI: 10.5829/idosi.mejsr.2013.18.6.12500.

[YSV15] Dash Yajnaseni, Kumar Sanjay, and Patle V.K. *A Survey on Serial and Parallel Optimization Techniques Applicable for Matrix Multiplication Algorithm.* Tech. rep. School of Studies inComputer Science &IT, Pt. RavishankarShukla University,Raipur, Chhattisgarh,492010, India., Jan. 2015. URL: https://www.researchgate.net/publication/285131617_A_Survey_on_Serial_and_Parallel_Optimization_Techniques_Applicable_for_Matrix_Multiplication_Algorithm (visited on 04/18/2025).

[DAm+16] Luisa D'Amore et al. "Mathematical Approach to the Performance Evaluation of Matrix Multiply Algorithm". en. In: *Parallel Processing and Applied Mathematics*. Ed. by Roman Wyrzykowski et al. Vol. 9574. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2016, pp. 25–34. ISBN: 978-3-319-32151-6 978-3-319-32152-3. DOI: 10.1007/978-3-319-32152-3_3. URL: http://link.springer.com/10.1007/978-3-319-32152-3_3 (visited on 04/18/2025).

[TW17] Thomas N. Theis and H.-S. Philip Wong. "The End of Moore's Law: A New Beginning for Information Technology". In: *Computing in Science & Engineering* 19.2 (Mar. 2017), pp. 41–50. ISSN: 1558-366X. DOI: 10.1109/MCSE.2017.29. URL: https://ieeexplore.ieee.org/abstract/document/7878935 (visited on 11/02/2025).

[GS20] Manuel Le Gallo and Abu Sebastian. "An overview of phase-change memory device physics". en. In: *J. Phys. D: Appl. Phys.* 53.21 (Mar. 2020). Publisher: IOP Publishing, p. 213002. ISSN: 0022-3727. DOI: 10.1088/1361-6463/ab7794. URL: https://iopscience.iop.org/article/10.1088/1361-6463/ab7794/meta (visited on 09/12/2025).

[Seb+20] Abu Sebastian et al. "Memory devices and applications for in-memory computing". en. In: *Nat. Nanotechnol.* 15.7 (July 2020). Publisher: Nature Publishing Group, pp. 529–544. ISSN: 1748-3395. DOI: 10.1038/s41565-020-0655-z. URL: https://www.nature.com/articles/s41565-020-0655-z (visited on 09/12/2025).

[Bru+21] Nazareno Bruschi et al. "GVSoC: A Highly Configurable, Fast and Accurate Full-Platform Simulator for RISC-V based IoT Processors". In: *2021 IEEE 39th International Conference on Computer Design (ICCD)*. arXiv:2201.08166 [eess]. Oct. 2021, pp. 409–416. DOI: 10.1109/ICCD53106.2021.00071. URL: http://arxiv.org/abs/2201.08166 (visited on 09/12/2025).

[Abb+23] Amir Abboud et al. *The Time Complexity of Fully Sparse Matrix Multiplication*. Version Number: 1. 2023. DOI: 10.48550/ARXIV.2309.06317. URL: https://arxiv.org/abs/2309.06317 (visited on 04/19/2025).

[He+23] Luchang He et al. "In-memory computing based on phase change memory for high energy efficiency". en. In: *Sci. China Inf. Sci.* 66.10 (Nov. 2023), p. 200402. ISSN: 1674-733X, 1869-1919. DOI: 10.1007/s11432-023-3789-7. URL: https://link.springer.com/10.1007/s11432-023-3789-7 (visited on 04/18/2025).

[Man+23] P. Mannocci et al. "In-memory computing with emerging memory devices: Status and outlook". en. In: *APL Machine Learning* 1.1 (Mar. 2023), p. 010902. ISSN: 2770-9019. DOI: 10.1063/5.0136403. URL: https://pubs.aip.org/aml/article/1/1/010902/2878744/In-memory-computing-with-emerging-memory-devices (visited on 09/12/2025).

[Ant+24] Alessio Antolini et al. "A Readout Scheme for PCM-Based Analog In-Memory Computing With Drift Compensation Through Reference Conductance Tracking". In: *IEEE Open Journal of the Solid-State Circuits Society* 4 (2024), pp. 69–82. ISSN: 2644-1349. DOI: 10.1109/OJSSCS.2024.3432468. URL: https://ieeexplore.ieee.org/document/10609348 (visited on 09/12/2025).

[BS24] Preet Bhutani and Amol Ashokrao Shinde. "Exploring the Impact of Multithreading on System Resource Utilization and Efficiency". In: *IJIREM* 11.5 (Oct. 2024), pp. 66–72. ISSN: 23500557. DOI: 10.55524/ijirem.2024.11.5.9. URL: https://ijirem.org/view_abstract.php?title=Exploring-the-Impact-of-Multithreading-on-System-Resource-Utilization-and-Efficiency&year=2024&vol=11&primary=QVJULTE4MjM= (visited on 04/19/2025).

[Gho+24] Amir Gholami et al. *AI and Memory Wall*. arXiv:2403.14123 [cs] version: 1. Mar. 2024. DOI: 10.48550/arXiv.2403.14123. URL: http://arxiv.org/abs/2403.14123 (visited on 09/12/2025).

[Kha+24] Asif Ali Khan et al. *The Landscape of Compute-near-memory and Compute-in-memory: A Research and Commercial Overview*. arXiv:2401.14428 [cs] version: 1. Jan. 2024. DOI: 10.48550/arXiv.2401.14428. URL: http://arxiv.org/abs/2401.14428 (visited on 09/12/2025).

[Tan+24] Tao Tang et al. "Multithreaded Reproducible Banded Matrix-Vector Multiplication". en. In: *Mathematics* 12.3 (Jan. 2024). Number: 3 Publisher: Multidisciplinary Digital Publishing Institute, p. 422. ISSN: 2227-7390. DOI: 10.

3390/math12030422. URL: https://www.mdpi.com/2227-7390/12/3/422 (visited on 04/18/2025).

[God] Matt Godbolt. *Compiler Explorer*. en. URL: https://godbolt.org/ (visited on 11/02/2025).